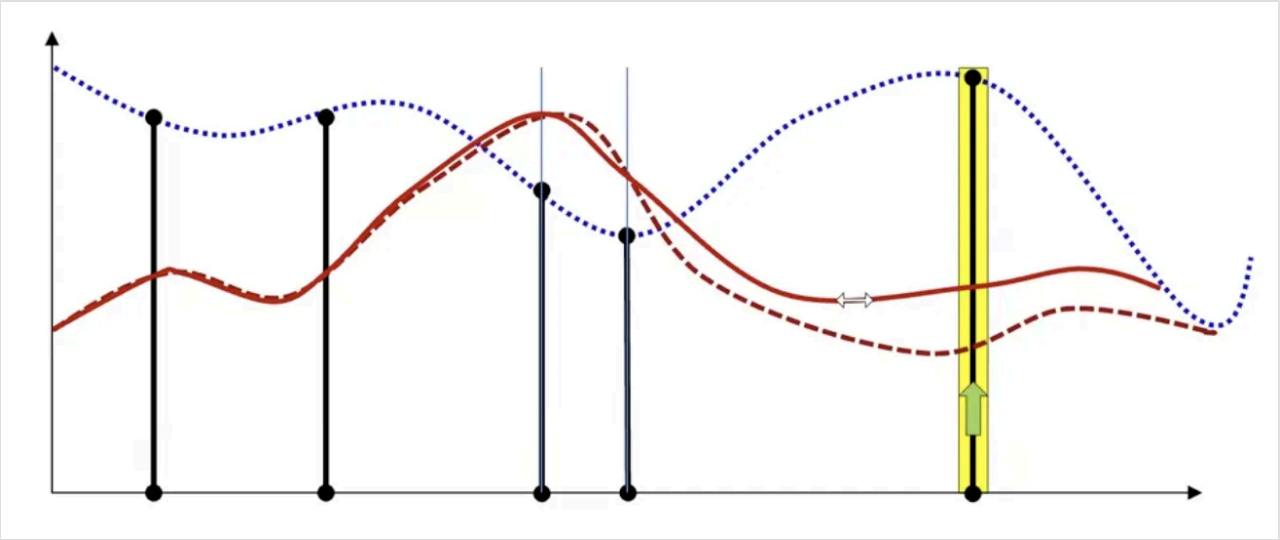
Lec7. Training Part V

Training Neural Networks : Optimization

Alternative : Incremental update



- 각 점마다, 따로 **training** 해감.
- Incremental update는 한 번에 하나의 샘플 gradient를 이용하여 파라미터를 점진적으로 갱신하고 전체 손실을 낮추는 것을 목표로 하지만, 각 업데이트에서 전체 경험적 손실이 반드시 감소하는 것은 아니다.

Given $(X_1, d_1), (X_2, d_2), \ldots, (X_T, d_T)$

Initialize all weights $W_1, W_2, \ldots, W_K$

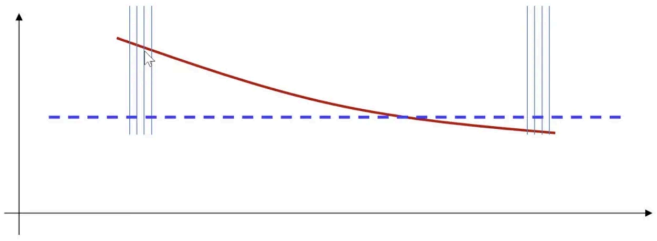
Do:

- For all $t = 1:T$
  - For every layer k:
    - Compute $\nabla_{W_k} \boldsymbol{Div}(\boldsymbol{Y_t}, \boldsymbol{d_t})$
    - Update

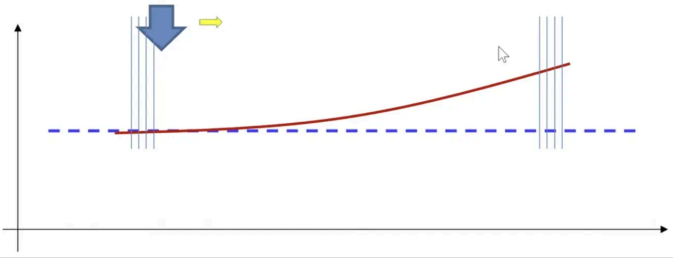
$$W_k = W_k - \eta \nabla_{W_k} \boldsymbol{Div}(\boldsymbol{Y_t}, \boldsymbol{d_t})^T$$

Until *Loss* has converged

Caveats: order of presentation



Caveats: order of presentation



- 이 경우, 위와 같이 iteration할 수 있음
- Incremental update에서는 한 번에 하나의 샘플로 파라미터를 갱신하며, 데이터가 제시되는 순서에 따라 학습 경로가 달라질 수 있다.
- 따라서 특정 순서에 반복적으로 영향을 받는 문제를 줄이기 위해 일반적으로 매 epoch마다 학습 데이터를 무작위로 섞어 사용한다.
- 다만 무작위 순서가 모든 incremental gradient 방법의 필수 조건인 것은 아니다.

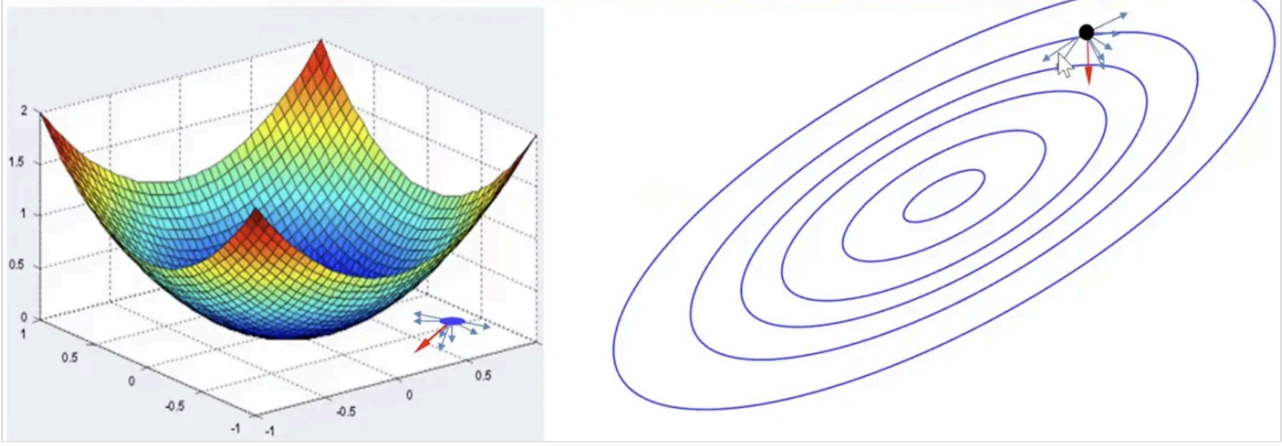
Do:

- Randomly permute $(X_1, d_1), (X_2, d_2), \ldots, (X_T, d_T)$
- For all $t = 1:T$
  - For every layer k:
    - Compute $\nabla_{W_k} \boldsymbol{Div}(\boldsymbol{Y_t}, \boldsymbol{d_t})$
    - Update

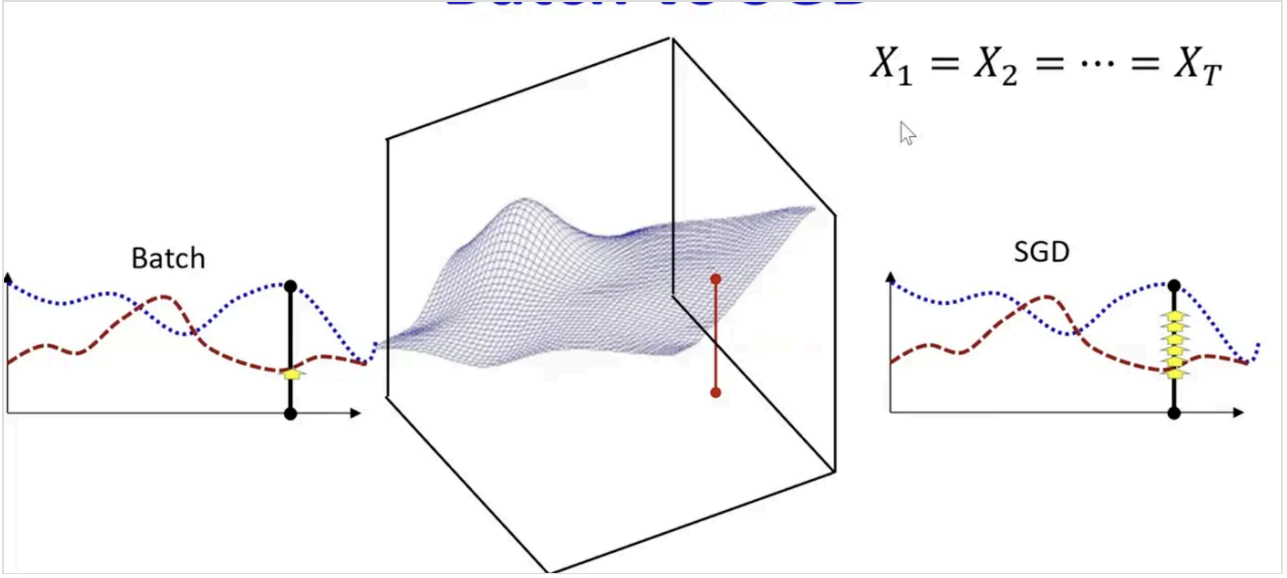
$$W_k = W_k - \eta \nabla_{W_k} \boldsymbol{Div}(\boldsymbol{Y_t}, \boldsymbol{d_t})^T$$

The expected behavior of the gradient

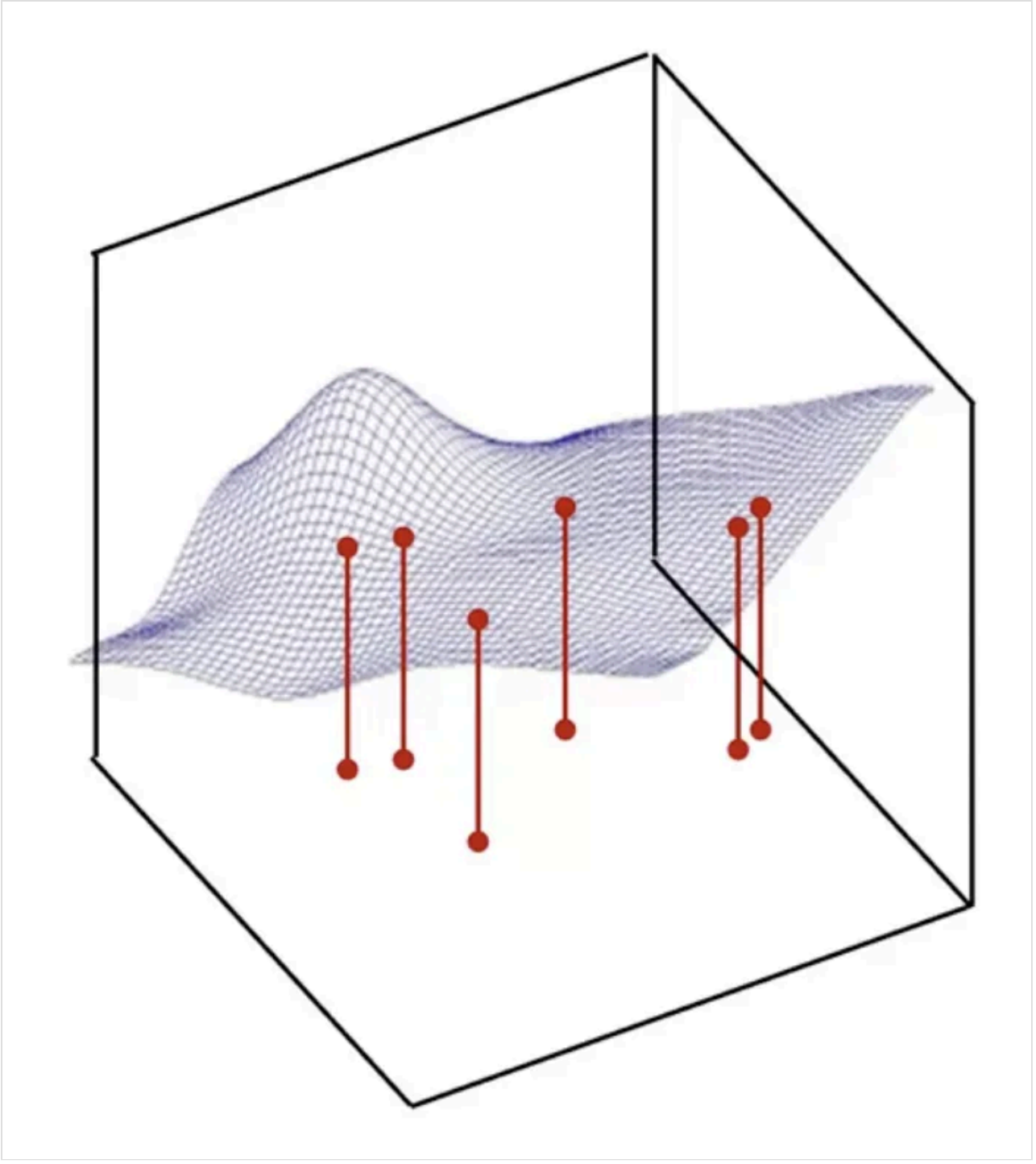
$$\frac{d\boldsymbol{E}(\boldsymbol{W}^{(1)}, \boldsymbol{W}^{(2)}, \ldots, \boldsymbol{W}^{(K)})}{d\boldsymbol{w}_{i,j}^{(k)}} = \frac{1}{\boldsymbol{T}} \sum_i \frac{d\boldsymbol{Div}(\boldsymbol{Y}(\boldsymbol{X}_i), \boldsymbol{d}_i; \boldsymbol{W}^{(1)}, \boldsymbol{W}^{(2)}, \ldots, \boldsymbol{W}^{(K)})}{d\boldsymbol{w}_{i,j}^{(k)}}$$



- 각 샘플의 손실 $L_i(W)$는 파라미터 W에 대한 gradient $g_i(W) = \nabla_W L_i(W)$를 만든다.
- 샘플별 gradient는 서로 같을 수도 있고 다른 방향을 가질 수도 있다.
- 전체 경험적 손실의 gradient는 동일한 파라미터에서 계산한 개별 gradient들의 평균인 $\nabla_W \text{Loss}(W) = (1/N) \Sigma_{i=1}^{N} g_i(W)$이며, SGD는 이 평균을 직접 계산하는 대신 무작위로 선택한 샘플의 gradient를 이용하여 파라미터를 갱신한다.



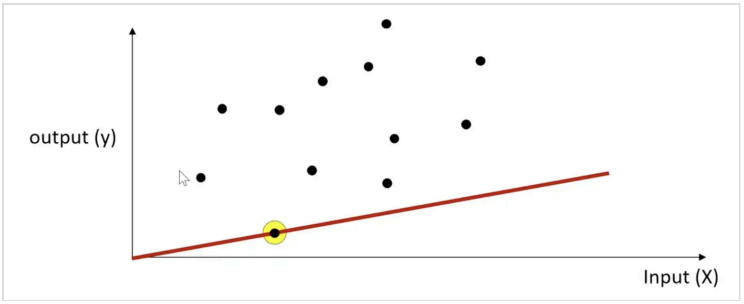
- 모든 데이터가 같은 곳에 뭉쳐있다면?
  - 모든 샘플이 유사한 입력과 타깃을 가지고, 현재 파라미터에서 계산한 샘플별 gradient도 서로 유사하다면 Batch gradient와 개별 SGD gradient의 방향이 비슷할 수 있다.
  - 그러나 입력 데이터의 평균으로 계산한 gradient가 개별 gradient들의 평균과 항상 같은 것은 아니다.
  - Batch gradient는 입력값의 평균이 아니라 각 샘플 손실의 gradient를 평균한 값으로 정의된다.
- Full-batch GD는 전체 N개 샘플의 gradient를 동일한 파라미터에서 계산하여 평균한 뒤 한 번 업데이트한다.
- 반면 SGD는 한 번에 하나의 샘플 gradient로 업데이트하므로 전체 데이터를 한 번 순회하는 동안 일반적으로 N번 업데이트한다.
- 두 방식 모두 한 epoch에서 전체 샘플을 처리할 수 있지만, SGD는 업데이트 사이에 파라미터가 변하며 실제 연산 시간도 배치 처리와 병렬화 방식에 따라 달라진다.
- 샘플별 gradient의 방향이 서로 다르면 평균 과정에서 일부 성분이 상쇄되어 Batch gradient의 크기가 작아질 수 있다.
- 그러나 데이터가 넓게 분포한다는 이유만으로 평균 gradient가 0에 가까워지는 것은 아니다.

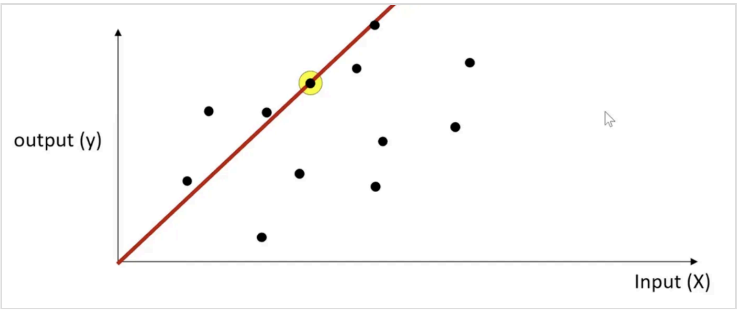


- Batch GD는 전체 경험적 손실의 gradient를 사용하고, SGD는 확률적인 샘플 gradient를 사용하므로 서로 다른 업데이트 경로를 가진다.
- SGD의 확률적 변동은 일부 비볼록 최적화 문제에서 탐색에 도움이 될 수 있으며, 작은 단위의 빈번한 갱신은 계산상 이점이 있을 수 있다.
- 다만 어느 방법이 더 효율적인지는 손실 함수, gradient의 분산, 학습률, 배치 크기 및 계산 환경에 따라 달라진다.

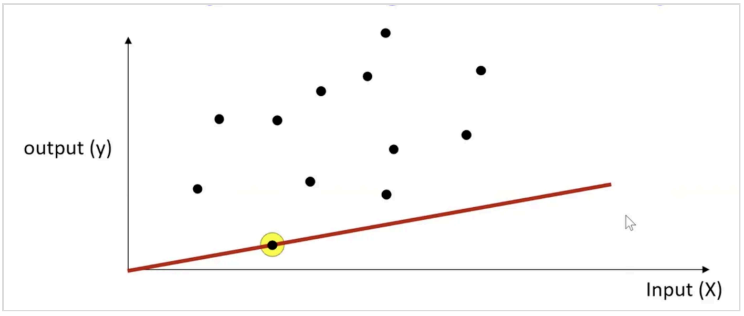
Incremental learning runs the risk of always chasing the latest input

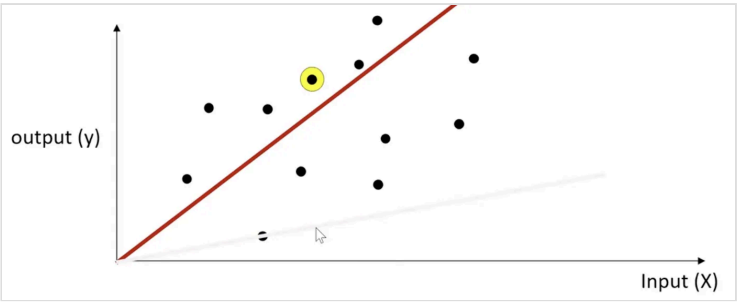
---



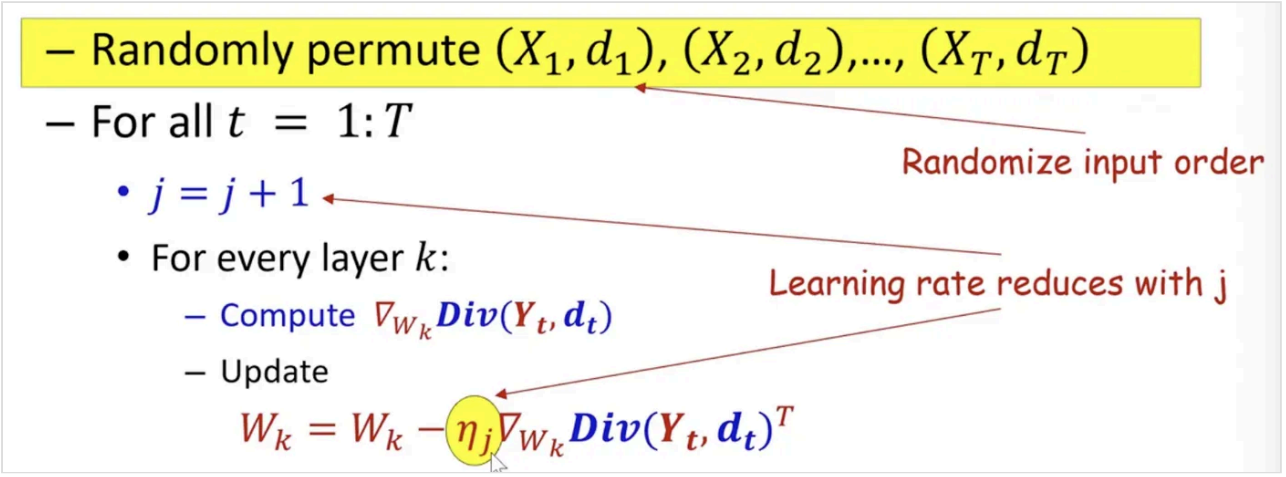


- SGD는 매 iteration마다 선택된 샘플의 손실 gradient를 이용하므로, 샘플별 gradient의 차이가 크면 파라미터가 최근 샘플의 gradient 방향에 민감하게 반응하고 업데이트 경로가 진동할 수 있다.
- 이러한 현상은 최신 샘플 추종 또는 확률적 변동으로 설명하는 것이 정확하며, 일반화 성능의 저하를 의미하는 과적합과는 구별해야 한다.
- 학습률을 점진적으로 감소시키면 후반부 업데이트의 변동을 줄일 수 있지만, 실제 수렴 여부는 학습률 조건과 목적함수 및 gradient의 성질에 의존한다.





학습률을 점진적으로 감소시키면 SGD의 확률적 업데이트 변동을 줄이는 데 도움이 될 수 있다. 다만 학습률을 감소시킨다는 사실만으로 수렴이 보장되지는 않으며, 실제 수렴 여부와 수렴 대상은 학습률 조건, 목적함수의 성질, gradient 추정의 비편향성 및 분산 등에 대한 추가 가정에 따라 달라진다.



SGD convergence (SGD 수렴 조건)

- Sufficient condition: step sizes follow the following conditions (Robbins and Munro 1951)

$$\sum_k \eta_k = \infty$$

  - Eventually the entire parameter space can be searched

$$\sum_k \eta_k^2 < \infty$$

  - The steps shrink
- The fastest converging series that satisfies both above requirements is

$$\eta_k \propto \frac{1}{k}$$

  - This is the optimal rate of shrinking the step size for strongly convex functions
- More generally, the learning rates are heuristically determined

- SGD에서는 샘플별 gradient가 서로 다른 경우 무작위 샘플링에 따른 확률적 gradient 추정 오차가 발생할 수 있다. 이러한 변동을 제어하는 고전적인 수렴 분석에서는 $\Sigma_{k=1}^{\infty} \eta_k = \infty$ 및 $\Sigma_{k=1}^{\infty} \eta_k^2 < \infty$를 만족하는 양의 학습률을 사용한다.
- 첫 번째 조건은 전체 이동 가능량이 지나치게 빨리 줄어드는 것을 막고, 두 번째 조건은 확률적 노이즈의 누적 영향을 제어하는 데 사용된다.
- 다만 이러한 조건은 학습률에 관한 조건일 뿐이며, 수렴 보장을 위해서는 문제에 맞는 목적함수의 성질과 gradient의 비편향성 및 분산 등에 대한 추가 가정이 필요하다.

1. Robbins-Monro Conditions

> SGD 수렴을 위한 학습률 조건:

- $\sum_{k=1}^{\infty} \eta_k = \infty$

> 학습률이 너무 빨리 감소하면 안 됨.
> 최적해까지 충분히 이동할 수 있어야 함.
> $\sum_{k=1}^{\infty} \eta_k^2 < \infty$

- 학습률을 충분히 빠르게 감소시켜야 함.
  확률적 Gradient에서 발생하는 노이즈의 누적 영향을 제한.

2. Learning Rate Schedule

- $\eta_k = \frac{1}{k^p}$

> 두 조건을 동시에 만족하려면:
> $\frac{1}{2} < p \leq 1$

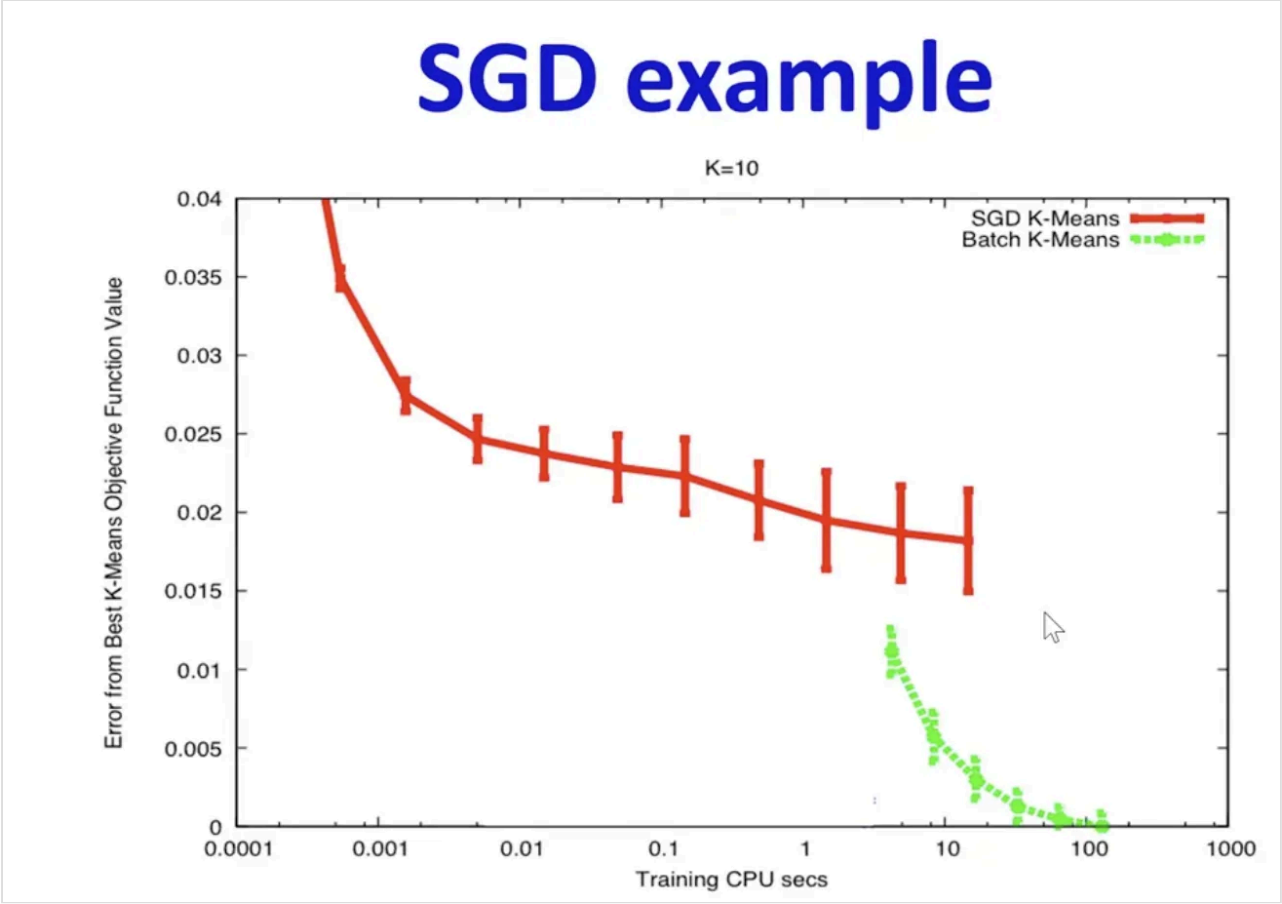
- 대표적으로 다음을 사용할 수 있음.
  $\eta_k \propto \frac{1}{k}$

3. Convex vs. Non-convex

> Convex Loss에서는 적절한 최적해 존재 조건과 SGD 수렴 가정하에 전역 최적해로의 수렴을 분석할 수 있다.

- Non-convex Loss에서는 일반적으로 전역 최적해를 보장할 수 없으므로, 목적함수가 매끄럽고 아래로 유계이며 확률적 gradient에 적절한 조건이 주어진 경우 gradient norm이 0에 가까워지는 정상성(stationarity)을 분석한다.
- 정상점은 gradient가 0인 지점으로 국소 최소점, 국소 최대점, 안장점을 포함할 수 있으며, gradient norm의 수렴과 파라미터 자체의 수렴은 구별해야 한다.

- SGD example



- 제시된 K-means 최적화 실험에서는 SGD 방식이 초기 계산 시간 동안 빠르게 개선되지만 결과의 변동성이 크고, Batch 방식은 더 긴 계산 시간을 사용하면서 최선의 K-means 목적함수 값과의 오차를 작게 만드는 모습을 보인다.
- 그림의 세로축은 목적함수의 절댓값이 아닌 최선의 결과와의 차이이므로, 값이 0에 가까워진다는 것은 실험에서 얻은 최선의 목적함수 값에 가까워진다는 뜻이다.
- 이 결과는 하나의 실험 사례이며 모든 최적화 문제에서 같은 상대적 성능을 보장하지는 않는다.

- Modelling a function

$$\hat{W} = \underset{W}{\operatorname{argmin}} \int_X div\big(f(X;W), g(X)\big) P(X) dX$$
$$= \underset{W}{\operatorname{argmin}}\, E\big[div\big(f(X;W), g(X)\big)\big]$$

- 최종적으로 minimize하고 싶은 것은 **integral of div / expected div**

$$Loss(W) = \frac{1}{N} \sum_{i=1}^{N} div(f(X_i;W), d_i)$$
$$\hat{W} = \underset{W}{\operatorname{argmin}}\, Loss(W)$$

> Empirical Risk Minimization (ERM)

- 최종 목표 : **Expected Risk (전체 데이터 분포의 평균 오차) 최소화**

Expected Risk

- R(W) = E

$$div(f(X;W), d)$$

  - → 실제 데이터 분포 전체에 대한 평균 Loss

Empirical Risk

- Loss(W) = (1/N) Σ_i div(f(X_i; W), d_i)
  - → 실제 분포를 알 수 없으므로, **학습 데이터의 평균 Loss로 근사**
  - → $\hat{W}$ = argmin Loss(W)
  - 즉, 평균 Loss를 최소화하는 W를 찾는 것.

SGD : Unbiased Estimate

- SGD : 매 iteration마다 **하나의 데이터만 선택하여 업데이트**
  - L_i(W) = div(f(X_i; W), d_i)

Sample Loss의 기댓값 = 전체 평균 Loss

- E

$$L_i\ (W)$$

= (1/N) Σ_i L_i(W) = Loss(W)

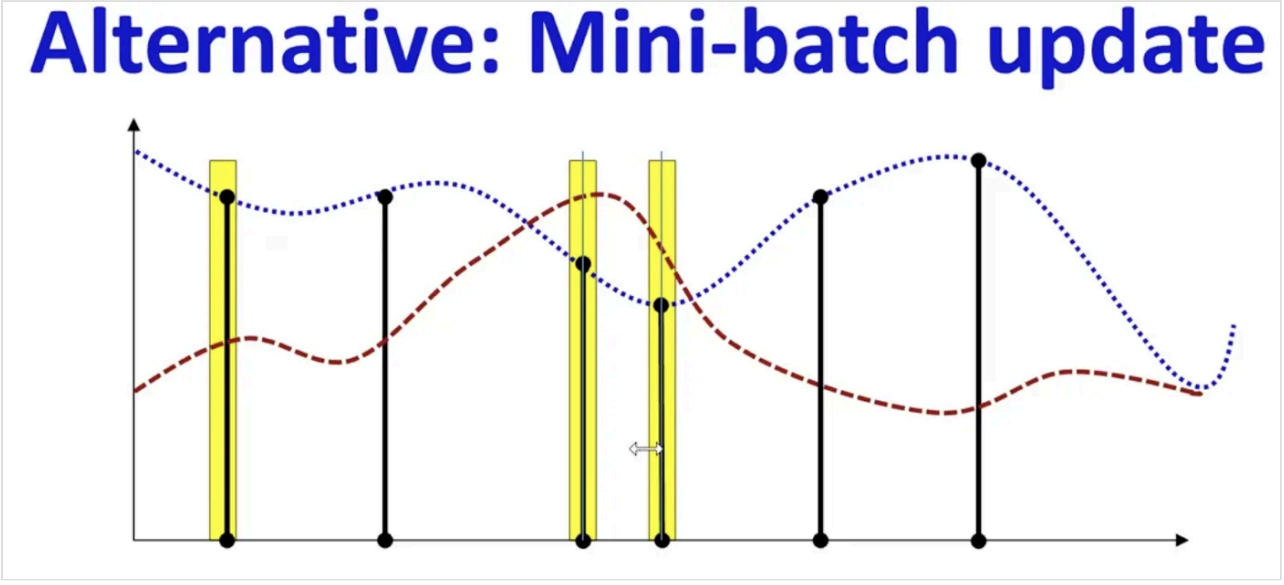
Gradient도 마찬가지

- E

$$\nabla L_i\ (W)$$

= ∇Loss(W)
  - → 이를 **Unbiased Estimate** 라고 함.
  - ⇒ 개별 샘플의 Gradient는 전체 경험적 Gradient와 다를 수 있지만, 전체 데이터에서 균등하게 무작위 추출하면 고정된 파라미터에서 계산한 샘플 Gradient의 기대값은 전체 경험적 Gradient와 같다.
- ⇒ 따라서 **전체 데이터를 매번 계산하지 않고도 SGD로 학습 가능.**
- 고정된 학습 데이터셋과 파라미터 W에 대해 샘플 인덱스 I를 전체 데이터에서 균등하게 선택하면, $E_I[L_I(W)]$ = Loss(W) 및 $E_I[\nabla L_I(W)]$ = ∇Loss(W)가 성립한다.
- 따라서 이 샘플링 방식의 SGD gradient는 전체 경험적 gradient의 비편향 추정량이다.
- 단, 복원 없는 순차 샘플링에서는 이미 선택한 샘플과 현재 파라미터를 조건으로 할 때 같은 비편향성이 항상 성립하지 않을 수 있으므로 샘플링 가정을 구별해야 한다.

SGD의 단점 해결 → Minibatch



- 무작위 n개를 선택해서, 학습에 이용함.

Given $(X_1, d_1), (X_2, d_2), \ldots, (X_T, d_T)$

Initialize all weights $W_1, W_2, \ldots, W_K$; $j = 0$

Do:

- Randomly permute $(X_1, d_1), (X_2, d_2), \ldots, (X_T, d_T)$
- For $t = 1 : b : T$
  - $j = j + 1$
  - For every layer k:
    - $\Delta W_k = 0$
  - For t' = t : t+b-1
    - For every layer k:
      - » Compute $\nabla_{W_k} Div(Y_t, d_t)$
      - » $\Delta W_k = \Delta W_k + \frac{1}{b} \nabla_{W_k} Div(Y_t, d_t)^T$
  - Update
    - For every layer k:

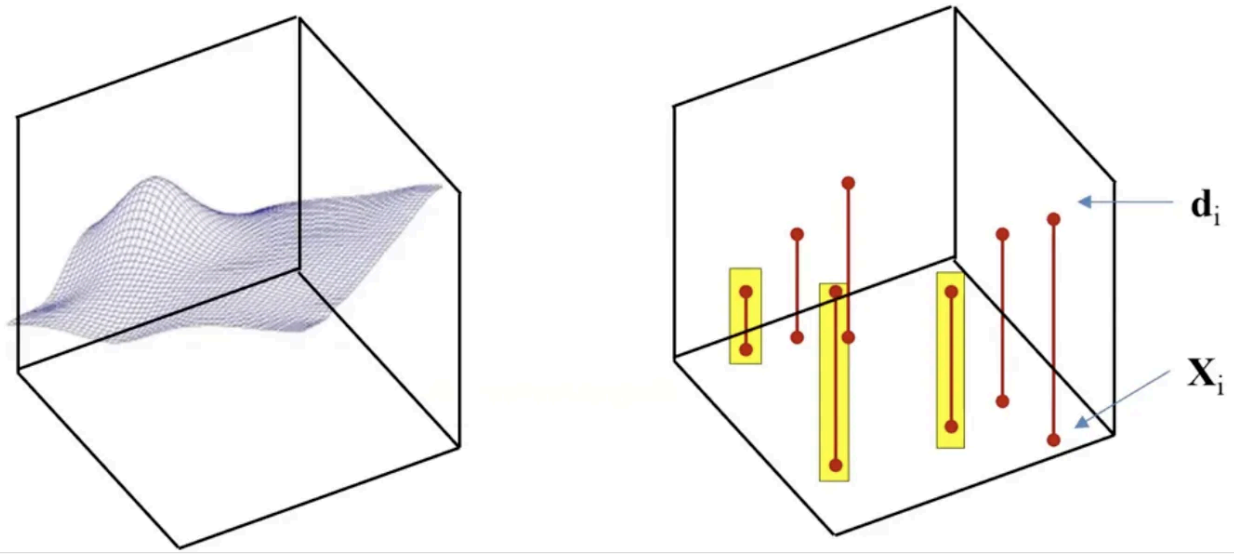
$$W_k = W_k - \eta_j \Delta W_k$$

Until Err has converged

Mini-batch size

Shrinking step size

Mini Batches



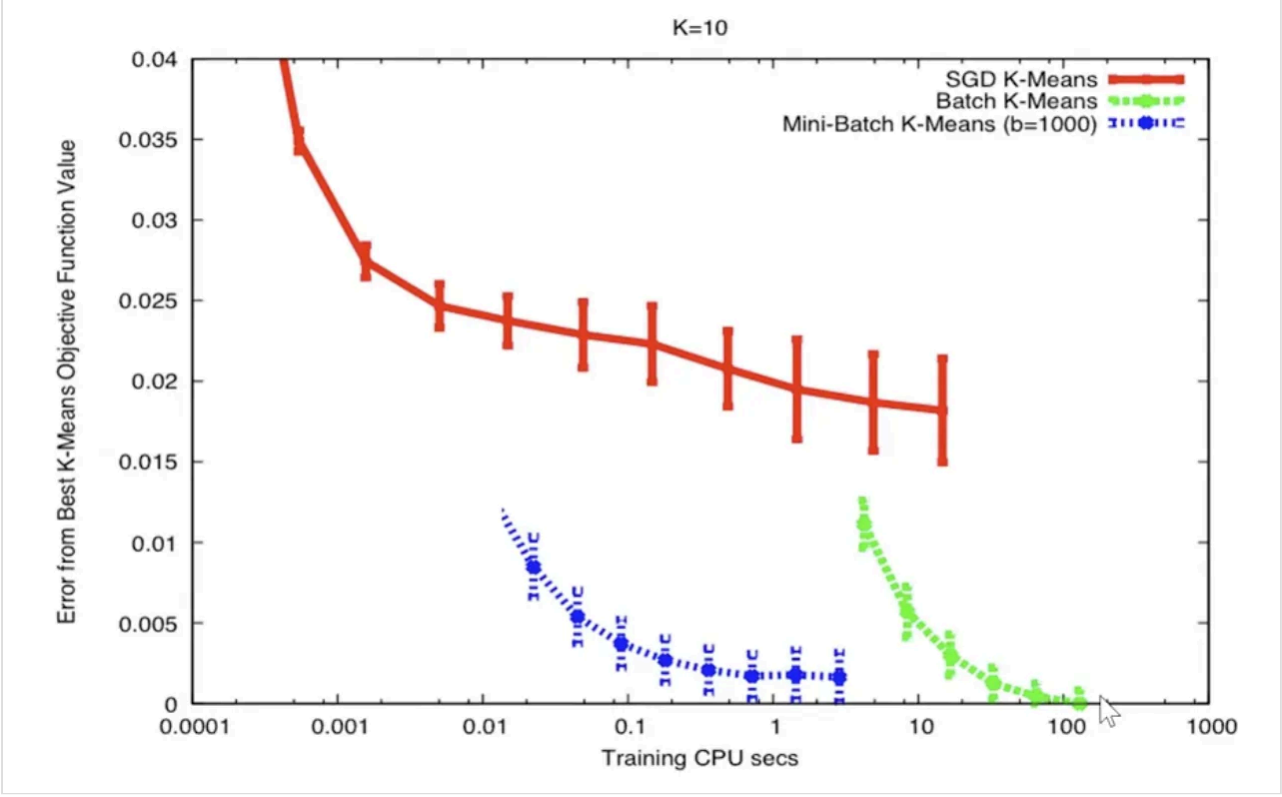
Mini-batch Loss

- $MiniBatchLoss(W) = \frac{1}{b} \sum_{i=1}^{b} div(f(X_i; W), d_i)$

Expected Mini-batch Loss

- $\mathbb{E}[MiniBatchLoss(W)] = \mathbb{E}[div(f(X; W), g(X))]$
- 고정된 학습 데이터셋 D에서 크기 b의 minibatch B를 균등하게 추출하면 $E_B[MiniBatchLoss_B(W)] = Loss_D(W)$가 성립하며, 모집단의 기대손실은 별도로 $R(W) = E_{(X,d)\sim P}[div(f(X; W), d)]$로 정의한다.

- 고정된 훈련 데이터셋 $D = \{(X_i, d_i)\}_{i=1}^{N}$에서 크기 b의 minibatch B를 균등하게 선택하고 $L_B(W) = (1/b)\ \Sigma_{i \in B}\ L_i(W)$로 정의하면, $E_B[L_B(W)] = (1/N)\ \Sigma_{i=1}^{N}\ L_i(W) = Loss(W)$가 성립한다.
- 이는 고정된 데이터셋에서 계산한 경험적 손실에 관한 등식이다.
- 한편 학습 데이터셋 D가 모집단 분포 P(X, d)로부터 독립적이고 동일하게 추출되고, 파라미터 W를 데이터셋 D와 무관하게 고정하면, 데이터 생성 과정에 대한 기대값을 취해 $E_D[Loss_D(W)] = R(W) = E_{(X,d)\sim P}[div(f(X; W), d)]$를 얻을 수 있다. 다만 학습 데이터셋 D로부터 선택한 파라미터 $\hat{W}(D)$에 대해서는 이 등식이 일반적으로 보장되지 않으며, 경험적 위험과 모집단 위험 사이에 일반화 격차가 발생할 수 있다.
- Mini-batch SGD는 크기 b의 무작위 배치에 포함된 gradient를 평균하여 파라미터를 갱신한다.
- 따라서 Batch GD와 단일 샘플 SGD 사이의 업데이트 특성을 가진다.
- 각 gradient를 독립적으로 복원 추출하고 분산이 유한하다고 가정하면 minibatch 평균 gradient의 분산은 단일 샘플 gradient 분산의 1/b배가 된다.
- 고정된 크기 N의 데이터셋에서 복원 없이 균등 추출할 경우에는 유한모집단 보정이 적용되며, b = N이면 전체 경험적 gradient를 정확히 계산하므로 샘플링 분산이 0이 된다.
- 여기서 감소하는 양은 개별 손실의 분산이 아니라 minibatch 평균 추정량의 분산이다.



Trend algorithms

RECALL : Momentum Update

$$\Delta W^{(k)} = \beta \Delta W^{(k-1)} - \eta \nabla_W Loss\left(W^{(k-1)}\right)^T$$
$$W^{(k)} = W^{(k-1)} + \Delta W^{(k)}$$

- Momentum은 이전 업데이트 방향을 일정 비율로 유지하면서 현재 gradient를 반영하여 업데이트 방향을 결정한다.
- $\Delta W^{(k)} = \beta \Delta W^{(k-1)} - \eta \nabla Loss(W^{(k-1)})$에서 β는 이전 업데이트의 영향이 얼마나 유지되는지를 결정하며 대표적으로 0.9를 사용할 수 있다.
- 이 업데이트는 과거 gradient의 지수 가중 누적합에 비례하지만, (1−β)로 정규화한 gradient의 지수이동평균과는 계수가 다르므로 동일한 식으로 취급해서는 안 된다.

RECALL : Nesterov's method

$$\Delta W^{(k)} = \beta \Delta W^{(k-1)} - \eta \nabla_W Loss\left(W^{(k-1)} + \beta \Delta W^{(k-1)}\right)^T$$
$$W^{(k)} = W^{(k-1)} + \Delta W^{(k)}$$

- Nesterov Momentum은 일반 Momentum과 달리 현재 파라미터가 아니라 이전 Momentum으로 이동할 것으로 예상되는 위치(look-ahead position)에서 gradient를 계산한다.
- 구체적으로 $\Delta W^{(k)} = \beta \Delta W^{(k-1)} - \eta \nabla Loss(W^{(k-1)} + \beta \Delta W^{(k-1)})$를 사용한 뒤 $W^{(k)} = W^{(k-1)} + \Delta W^{(k)}$로 갱신한다.
- 이러한 방식은 Momentum에 따른 선행 이동을 고려하여 gradient를 평가한다는 점에서 일반 Momentum과 구별된다.

RMSProp (Root Mean Square Propagation)

- 각 파라미터마다 **Learning Rate를 다르게 조절**하는 최적화 알고리즘.

1. 아이디어

- RMSProp은 각 파라미터 좌표에 대해 gradient 제곱의 지수이동평균 v_t를 유지하고, 이를 이용해 gradient를 좌표별로 정규화한다.
- 과거 gradient 제곱의 이동평균이 큰 좌표에는 상대적으로 작은 유효 학습률이 적용되고, 이동평균이 작은 좌표에는 상대적으로 큰 유효 학습률이 적용된다.

2. 수식

- Gradient 제곱의 이동평균:

  - $v_t = \beta v_{t-1} + (1-\beta) g_t^2$
- 파라미터 업데이트:
  - $w_{t+1} = w_t - \eta g_t / (\sqrt{v_t} + \varepsilon)$
- ※ g_t^2는 Gradient의 제곱이지, 2차 미분이 아님.
- 여기서 g_t는 현재 파라미터에서 계산한 gradient 또는 minibatch gradient 추정값이고, $0 \leq \beta < 1$, $v_0 = 0$, $\eta > 0$, $\varepsilon > 0$로 두며, g_t^2는 2차 미분이 아닌 gradient의 원소별 제곱이고 제곱근과 나눗셈도 좌표별로 계산한다.
- 이로 인해 좌표별 gradient 크기의 차이를 완화하고 일부 학습 문제에서 업데이트를 안정화할 수 있지만, 모든 문제에서 일반 SGD보다 좋은 수렴 성능이 보장되는 것은 아니다.

Adam (Adaptive Moment Estimation)

Momentum + RMSProp 결합

- Adam은 gradient의 1차 모멘트와 제곱 gradient의 2차 원시 모멘트를 지수이동평균으로 추정하고, 이를 이용해 좌표별 업데이트 크기를 조절한다.

1. Momentum (Gradient 이동평균)

- $m_k = \delta m_{k-1} + (1-\delta) g_k$
  → 업데이트 방향 안정화
- 초기값은 $m_0 = v_0 = 0$으로 두고, $k = 1, 2, \cdots$에서 $m_k = \delta m_{k-1} + (1-\delta) g_k$, $v_k = \gamma v_{k-1} + (1-\gamma) g_k^2$를 계산한다.
- 여기서 $0 \leq \delta, \gamma < 1$이며 제곱은 원소별 연산이다.

2. RMSProp (Gradient 제곱 이동평균)

- $v_k = \gamma v_{k-1} + (1-\gamma) g_k^2$
  - → 파라미터별 Learning Rate 조절

3. Bias Correction

- $\hat{m}_k = m_k / (1-\delta^k)$
  $\hat{v}_k = v_k / (1-\gamma^k)$
  - → 초기 이동평균의 편향 보정

4. Parameter Update

- $w_{k+1} = w_k - \eta \hat{m}_k / (\sqrt{\hat{v}_k} + \varepsilon)$
  - Adam은 좌표별 유효 업데이트 스케일을 적응적으로 조절하지만 기본 학습률 η는 별도로 설정해야 하며, 모든 문제에서 더 빠른 수렴을 보장하지는 않는다.